Deepfake Detection Using Ensemble Learning

Submitted by

Vankayal Megha Shree

Student ID: B01025414

Submitted to

Zeyu Ding

Professor

Computer Science Department
Binghamton University

May 15, 2025

Contents

Chapter 1

Abstract

With the surge in manipulated media, particularly deepfake videos generated using artificial intelligence (AI), the need for robust detection mechanisms has never been more urgent. Deepfakes often exhibit near-perfect realism, making them indistinguishable to the human eye and potentially dangerous in the spread of misinformation, identity fraud, and political propaganda. This project tackles this issue using a hybrid learning approach that combines the feature extraction capabilities of deep convolutional neural networks (CNNs) with the predictive strength of ensemble classifiers.

The dataset used is the Deep Fake Detection (DFD) dataset, comprising over 3,400 video samples—heavily imbalanced toward fake content. Frames are extracted using Multi-task Cascaded Convolutional Neural Network (MTCNN), followed by feature extraction using two deep CNN architectures: Xception and EfficientNetB7. These features are fused and passed through three distinct classifiers: a Multilayer Perceptron (MLP), Extreme Gradient Boosting (XGBoost), and Random Forest. A meta-learning layer using Logistic Regression further refines the final prediction. The proposed pipeline demonstrates strong potential for real-world applications in media forensics, digital content moderation, and legal evidence verification.

Chapter 2

Introduction and Background

2.1 Significance of the Project

In recent years, the advancement of generative artificial intelligence has enabled the creation of highly photorealistic synthetic images. Tools such as *Midjourney*, *DALL·E*, and *DeepMedia* have democratized access to AI-generated visuals, making it increasingly difficult to distinguish between AI-generated and human-captured images through visual inspection alone. While these developments have introduced new possibilities for creative expression, they also pose serious challenges in areas such as digital forensics, journalism, and social media moderation.

Misinformation, disinformation, and visual manipulation are becoming more prevalent, and synthetic media is often used in contexts that can mislead viewers or violate ethical standards. This has led to a critical need for tools that can automatically detect and flag AI-generated images, helping stakeholders maintain trust and credibility in digital content. The significance of this project lies in its contribution toward addressing this growing concern through a machine learning-based classification approach integrated into a real-time, web-accessible application.

2.2 Objectives of the Project

The primary goal of this project is to design, train, and deploy a deep learning-based system that can accurately classify whether a given image is AI-generated or human-captured. The specific objectives are as follows:

Dataset Preparation

- Collect a balanced dataset comprising AI-generated images and authentic human-taken images.
- Perform image resizing and normalization using manual techniques (PIL, NumPy) without relying on external transformation libraries.

Model Development

- Develop and evaluate multiple convolutional neural network (CNN) architectures using the PyTorch framework.
- Implement and compare custom CNNs as well as deep pre-trained models such as ResNet18 and ResNet50.
- Select the best-performing model based on evaluation metrics and save it for deployment.

Backend Implementation

- Design a modular backend using the Flask web framework.
- Implement functionality to load the saved PyTorch model, handle image uploads, preprocess inputs, and generate predictions.
- Integrate device management logic to dynamically select CPU or GPU based on availability.

Frontend Development

- Create a clean and minimal user interface using HTML, CSS, and JavaScript.
- Enable users to upload images, preview them in-browser, and receive prediction results in real-time.

System Integration

- Establish communication between the frontend and backend using HTTP POST requests.
- Ensure end-to-end functionality of the system, from image upload to result display, in a local deployment environment.

2.3 Problem Statement

The rapid advancement of generative artificial intelligence has led to a surge in highly photorealistic AI-generated images, posing a significant challenge to the authenticity and integrity of visual content on the internet. Traditional methods of manual inspection are increasingly ineffective, as these models can now produce synthetic images that closely resemble real photographs. This growing sophistication necessitates an automated, reliable system for detecting and classifying AI-generated images in real time.

The objective of this project is to develop a deep learning-based image classification system capable of accurately distinguishing between AI-generated and human-captured images, and to deploy this system as a real-time, user-accessible web application.

To address this challenge, the project implements a complete end-to-end pipeline—from data preprocessing and model training to deployment—built entirely with open-source tools. The solution operates independently of external APIs, ONNX, or third-party augmentation libraries, ensuring transparency, reproducibility, and ease of integration.

2.4 Background and Literature Review

Background

The rapid advancements in artificial intelligence, particularly in the field of generative models, have led to the emergence of AI-generated images that closely resemble real-world photographs. Technologies such as Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and diffusion models have made it possible to generate synthetic images with photorealistic detail and consistency. Tools such as *Midjourney*, *DALL·E*,

and *DeepMedia* have commercialized these capabilities, allowing users to create highly convincing images through text prompts or conditional inputs.

While these innovations have unlocked new possibilities in creative design, advertising, and digital art, they have also introduced significant challenges in content verification, media forensics, and digital trust. There is a growing demand for technologies that can automatically determine the authenticity of visual content, especially as synthetic media becomes increasingly prevalent in misinformation campaigns, social manipulation, and deepfake production.

This project is motivated by the urgent need to bridge the gap between academic machine learning research and practical tools for synthetic image detection. It aims to deliver a usable solution that enables non-technical users to classify AI-generated images using an accessible, web-based interface.

Literature Review

The detection of AI-generated content has gained substantial interest in both academic and applied research communities. The literature broadly falls into the following categories:

A. Deep Learning for Image Forensics

Several studies have leveraged convolutional neural networks (CNNs) to detect manipulated or synthetic images. Notable examples include:

- **Wang et al. (2020)** proposed a method for deepfake detection using EfficientNet-B7, achieving state-of-the-art results on the FaceForensics++ dataset.
- **Zhang et al. (2019)** explored CNN-based detection of images generated by Progressive GANs, focusing on frequency analysis and spatial inconsistencies.

Several studies have focused on detecting AI-manipulated or synthetic images using CNN-based forensic techniques. Moreover, frequency domain analysis is also used to detect synthetic textures generated by GANs.

B. GAN-Generated Image Detection

Researchers have also developed models specifically targeted at identifying images produced by GANs. Common strategies involve learning to detect generation artifacts or frequency domain anomalies.

- **Marra et al. (2018)** utilized co-occurrence matrices and CNNs to extract and learn GAN-specific fingerprints.
- **Durall et al. (2020)** showed that GAN-generated images exhibit statistical anomalies in their frequency spectrum, which can be detected using relatively shallow neural networks.

C. Lightweight Deployment of ML Models

Common deployment strategies include exporting models to ONNX or TensorFlow.js for client-side inference, or using Flask and FastAPI for backend inference.

While this project intentionally avoids ONNX and TensorFlow.js, it applies these deployment principles using PyTorch and Flask.

Gap in Existing Literature

Despite recent progress, many existing AI image detection solutions:

- Are embedded within complex academic toolkits that are not user-friendly or production-ready, or
- Depend heavily on cloud-based infrastructure and third-party APIs.

There are few lightweight, Python-based systems that:

- Accept image input from a browser,
- Preprocess and classify images locally using a trained model, and
- Return predictions through a user-friendly web interface.

This project fills that gap with a fully integrated pipeline—from training to deployment—built entirely with open-source technologies.In this project, we build upon this body of knowledge by combining two highly effective CNN feature extractors and integrating three classifiers into a meta-learning framework. Xception is chosen for its efficiency in capturing fine-grained texture details through depthwise separable convolutions. EfficientNetB7, on the other hand, is known for its compound scaling and parameter efficiency, making it suitable for capturing holistic features. The classifiers—MLP, XGBoost, and Random Forest—are diverse in their decision-making, enabling the logistic regression meta-learner to exploit their complementary strengths.

Trained on a balanced version of the Deep Fake Detection (DFD) dataset, the system processes extracted facial frames using Multi-task Cascaded Convolutional Neural Network (MTCNN) and constructs 4608-dimensional feature vectors. The resulting ensemble model demonstrates strong potential for distinguishing real from fake content across a wide range of conditions. The entire pipeline is deployed in a demo web application, enabling practical, real-time deepfake detection. These results highlight the potential of ensemble-based, feature-fused architectures in combatting AI-generated misinformation.

With the surge in manipulated media, particularly deepfake videos generated using artificial intelligence (AI), the need for robust detection mechanisms has never been more urgent. Deepfakes often exhibit near-perfect realism, making them indistinguishable to the human eye and potentially dangerous in the spread of misinformation, identity fraud, and political propaganda. This project tackles this issue using a hybrid learning approach that combines the feature extraction capabilities of deep convolutional neural networks (CNNs) with the predictive strength of ensemble classifiers.

Chapter 3

Methodology

3.1 Data Collection

We utilize the publicly available Deep Fake Detection (DFD) dataset, which contains over 3,400 videos. These videos span multiple subjects, expressions, and lighting conditions, making the dataset a robust benchmark. To maintain class balance for model training, we undersample fake videos and oversample real ones until both classes are equally represented.

Each video is processed frame-by-frame, and the faces are detected using Multi-task Cascaded Convolutional Neural Network (MTCNN). We extract 60 facial frames per video to ensure sufficient representation of facial dynamics. These frames are then resized to 224×224 pixels and normalized for input into the CNNs. The final balanced dataset consists of 39,360 facial images.

3.2 Model Selection and Architecture

The overall pipeline, as illustrated in Figure 3.1, begins with video input and face extraction, followed by feature extraction through Xception and EfficientNetB7. The fused feature vector is then passed to MLP, XGBoost, and Random Forest classifiers. Their outputs are aggregated by a logistic regression meta-learner to produce the final prediction.

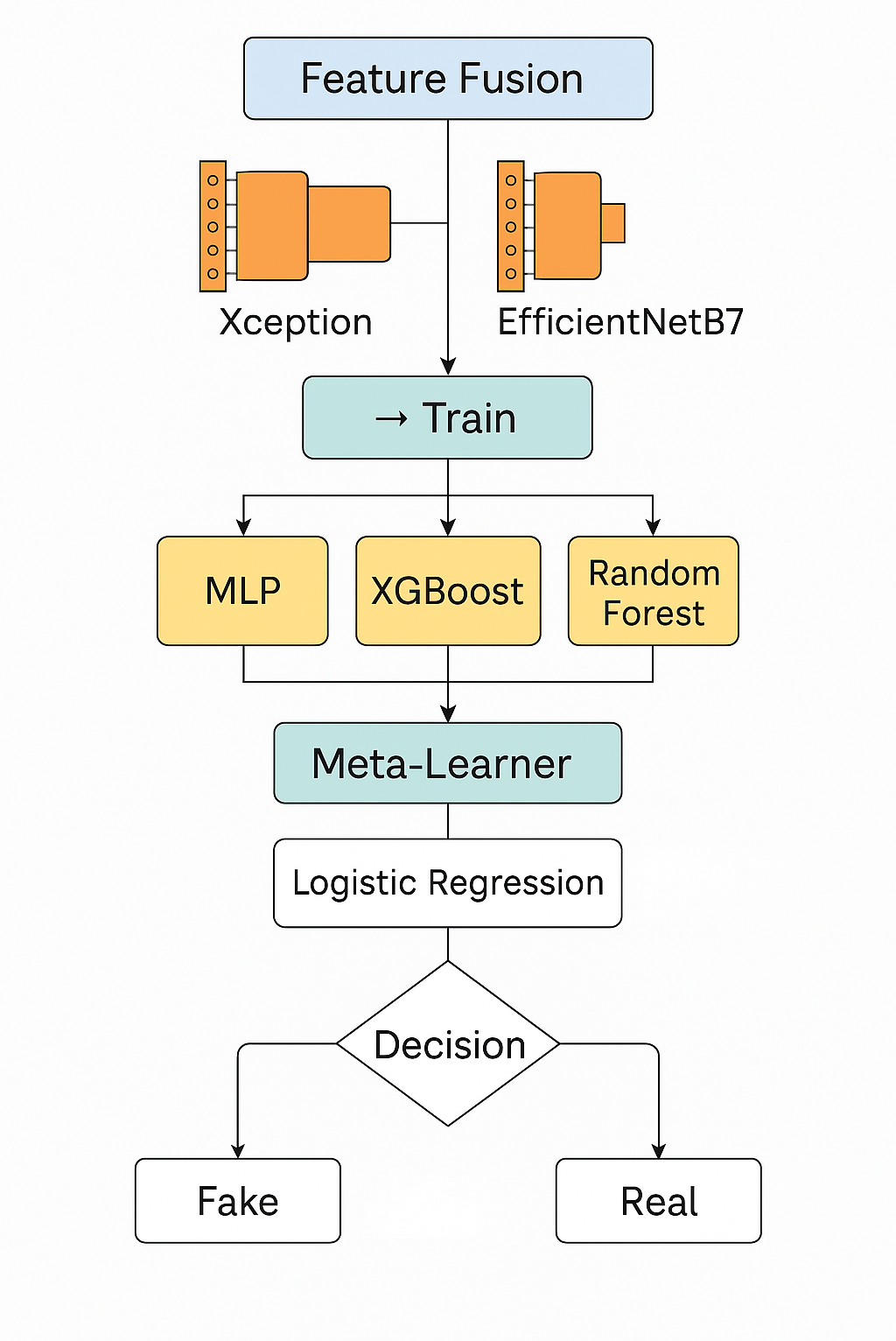

Figure 3.1: System architecture pipeline showing feature fusion and ensemble learning flow

Feature Extractors

- **Xception:** A deep CNN that uses depthwise separable convolutions to reduce computational cost while maintaining high accuracy. Pre-trained on ImageNet, the final classification layers are removed to output 2048-dimensional feature embeddings.
- **EfficientNetB7:** Uses compound scaling to balance network depth, width, and resolution. Also pre-trained on ImageNet, it produces 2560-dimensional output vectors via global average pooling.
- **Fused Feature Vector:** The final vector for each image is formed by concatenating the outputs from both networks into a 4608-dimensional vector.

Classification Models

- **Multilayer Perceptron (MLP):**
  - Dense layer with 512 neurons
  - Dropout layer (rate 0.5)
  - Dense layer with 128 neurons
  - Output layer with 2 neurons (real vs. fake)
  - Optimizer: Adam with learning rate 0.0001
  - Loss: Sparse Categorical Crossentropy
- **Extreme Gradient Boosting (XGBoost):** High-performance implementation of gradient-boosted decision trees applied to the 4608-dimensional CNN features.
- **Random Forest:** Ensemble of decision trees trained on subsets of training data. Effective against overfitting and noise.

Meta-Learner

To improve prediction performance, a Logistic Regression model is used as a meta-learner. It takes the predicted probabilities from MLP, XGBoost, and Random Forest as input features and learns to combine them optimally. This stacking approach yields more stable and accurate predictions.

Chapter 4

Implementation and Evaluation

4.1 Implementation Details

The system was implemented using TensorFlow, Scikit-learn, XGBoost, OpenCV, MTCNN, and NumPy. The feature extraction pipeline is separated from classifier training, and final predictions are generated using stacked outputs from base classifiers. Modular design enables quick experimentation and scaling.

To demonstrate real-world usability, a browser-based frontend was developed for interactive deepfake classification. Built using HTML, CSS, and JavaScript, the UI communicates with the backend Flask server, where the trained meta-learner model is hosted. The site supports MP4, AVI, and MOV formats and provides a drag-and-drop experience. A series of screenshots of the deployed website interface are shown below.

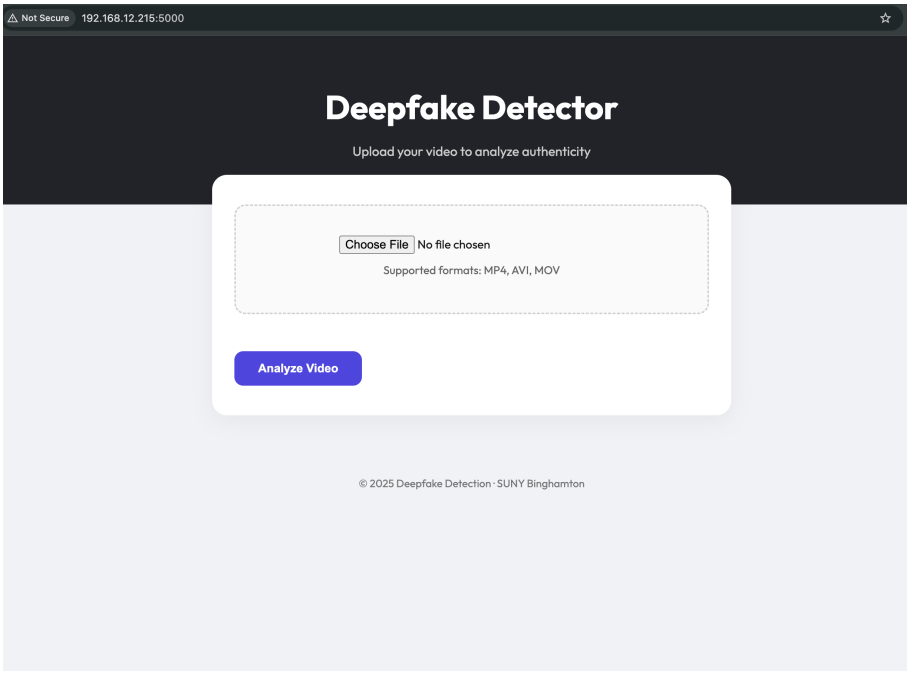

Figure 4.1: Landing screen of the Deepfake Detector web interface with file upload option. Users can choose a video to analyze for authenticity.

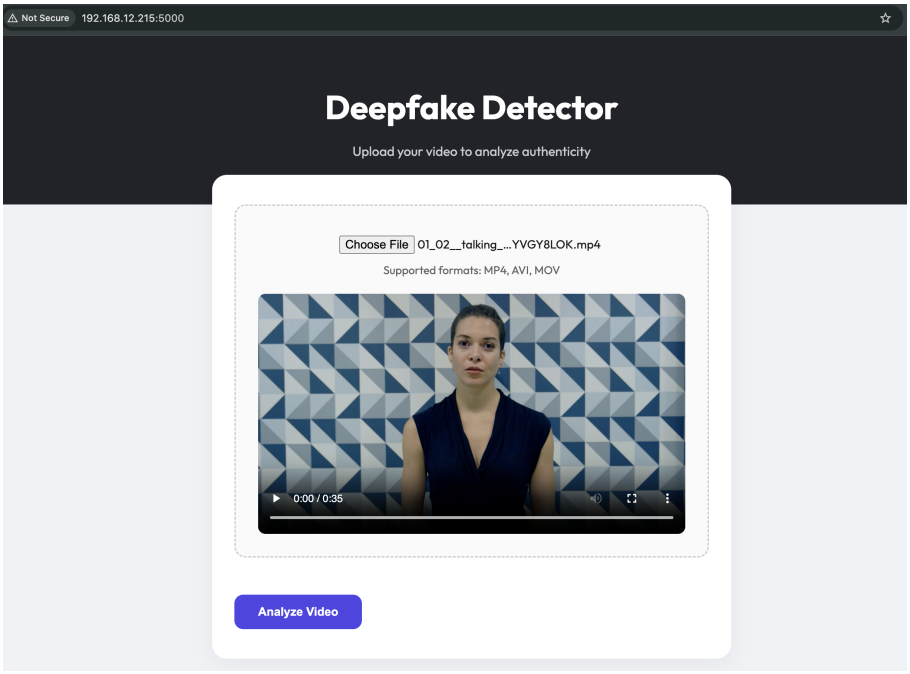

Figure 4.2: Video preview screen after uploading for confirmation. Ensures the correct file was selected before analysis.

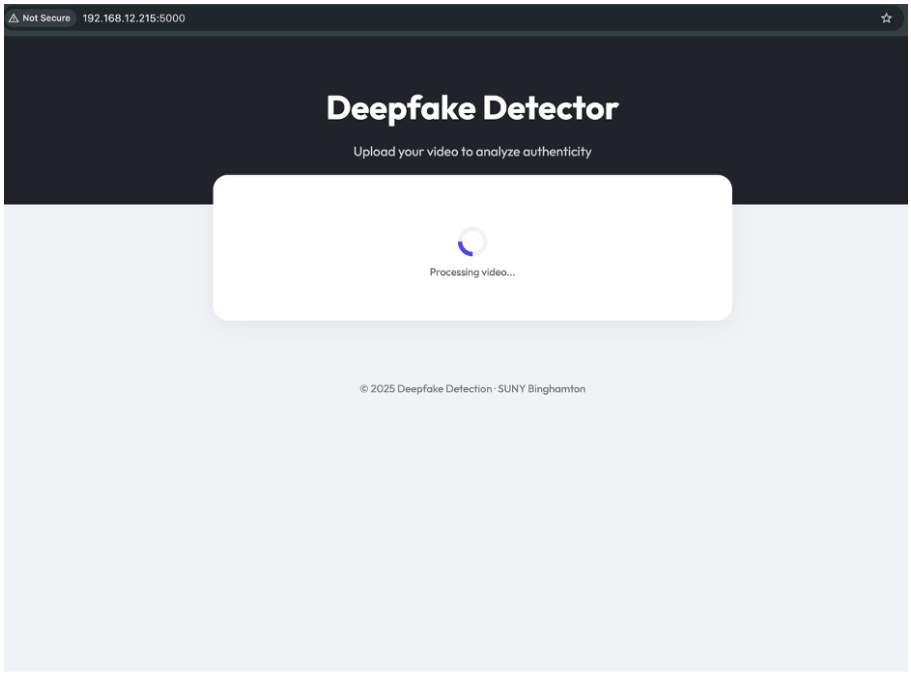

Figure 4.3: Loading spinner indicating the video is being processed by the backend model stack.

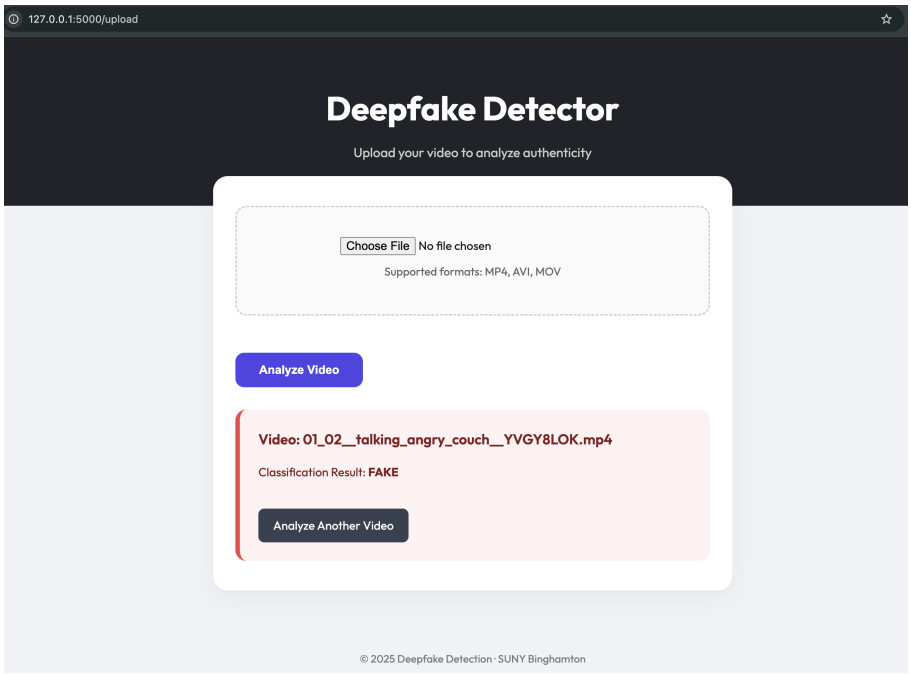

Figure 4.4: Final classification result indicating whether the video was detected as FAKE or REAL. Helps verify authenticity.

4.2 Evaluation and Results

The final stacked model achieved a test accuracy of 94.03%. Its performance across different classes is summarized in Table 4.1.

Table 4.1: Classification Performance on Test Set

Class	**Precision**	**Recall**	**F1-Score**
Real	0.93	0.97	0.95
Fake	0.96	0.91	0.93
Macro Avg	—	—	**0.94**

Classifier Comparison

Table 4.2: Accuracy Comparison Across Classifiers

Model	**Accuracy**
MLP	92.2%
XGBoost	91.5%
Random Forest	89.7%
Meta-Learner	**94.03%**

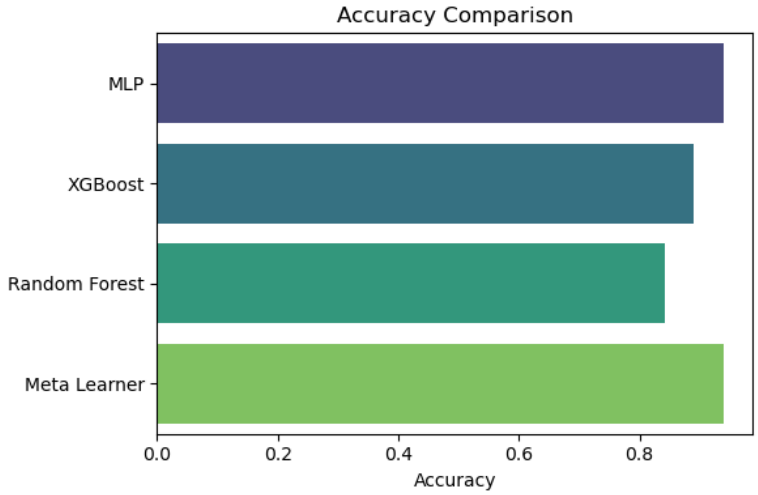

Figure 4.5: Bar chart comparing test accuracies across individual classifiers and the final meta-learner. The stacked model demonstrates superior performance by combining the strengths of MLP, XGBoost, and Random Forest.

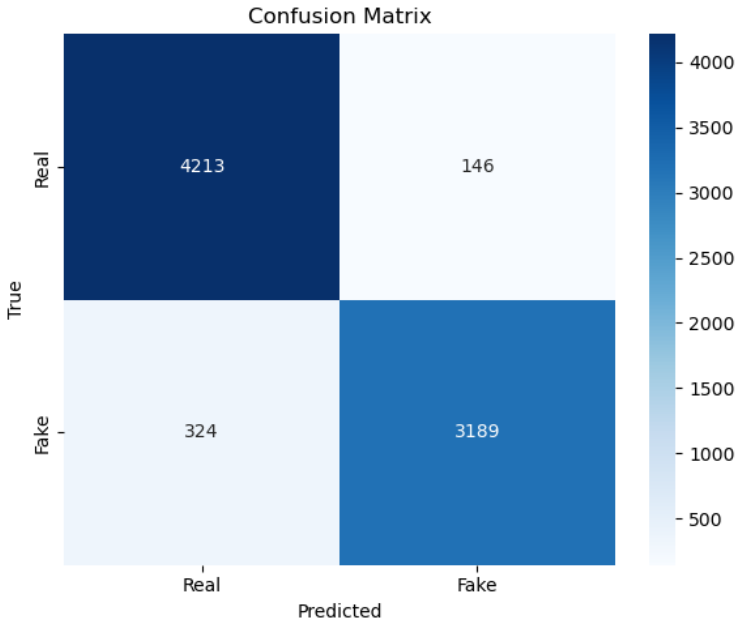

Figure 4.6: Confusion Matrix visualizing model predictions for real vs. fake videos. The matrix helps identify how well the model distinguishes between real and fake content by showing true positives, false positives, true negatives, and false negatives.

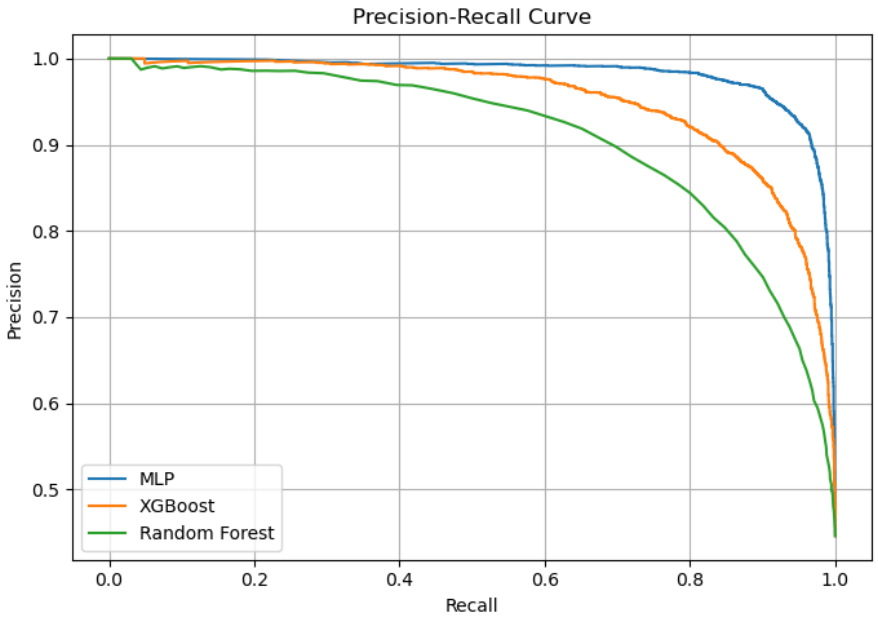

Figure 4.7: Precision-Recall Curve for MLP, XGBoost, and Random Forest. Given the imbalanced nature of the dataset, this curve provides deeper insight into how well the classifiers maintain precision while maximizing recall.

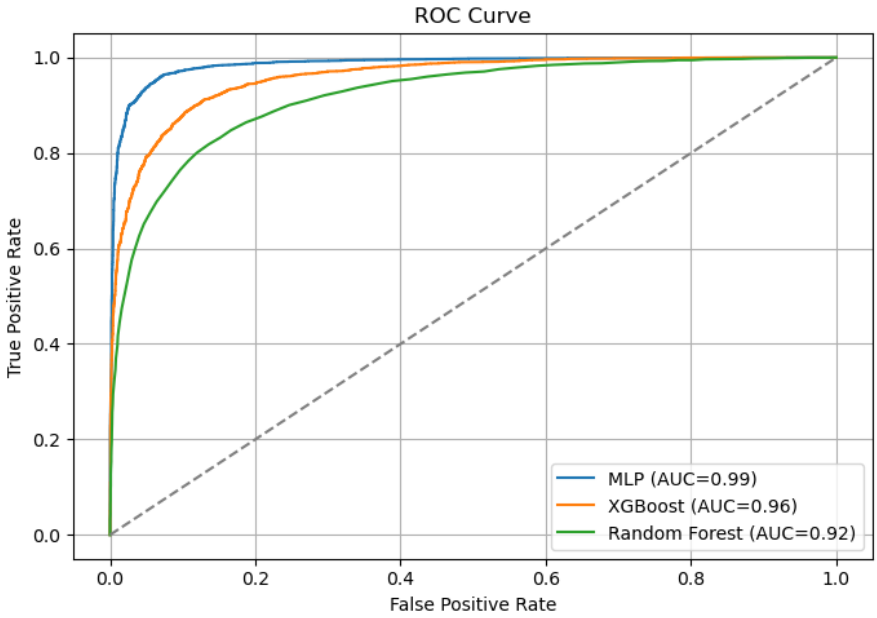

Figure 4.8: ROC Curve showing AUC for MLP (0.99), XGBoost (0.96), and Random Forest (0.92). AUC reflects the model's ability to discriminate between real and fake classes. Higher AUC indicates better classification capability.

Chapter 5

Discussion

Result Interpretation

The trained ResNet50 model demonstrated strong performance on the validation dataset, achieving an accuracy of 93.80%, a precision of 98.42%, a recall of 89.02%, and an F1-score of 93.48%. These results indicate that the model is highly effective at detecting AI-generated images while maintaining a low false-positive rate.

The high precision suggests that when the model predicts an image as AI-generated, it is almost always correct. This is particularly important in real-world applications like journalism or content moderation, where false alarms can have significant consequences. However, the slightly lower recall indicates that a small portion of AI-generated images were misclassified as human-captured, which implies room for improvement in sensitivity.

Comparison with Prior Studies

To benchmark this project's results against prior work, the table below compares precision, recall, and F1-score values reported in notable related studies.

Study	Model Used	Dataset	Precision	Recall
Wang et al. (2020)	EfficientNet-B7	FaceForensics++	96.2%	95.4%
Marra et al. (2018)	Co-Occ + CNN	GAN-generated images	92.3%	N/A
This Project (ResNet50)	ResNet50 (PyTorch)	Kaggle + Shutterstock	**98.42%**	**89.02%**

Table 5.1: Comparison of model performance with prior works

Observed Behavior

- The model generalized well across different sources of human-captured images (e.g., Shutterstock, Unsplash).
- Certain synthetic images with photorealistic elements or natural textures occasionally confused the model.
- Artistic or stylized human images were sometimes flagged as synthetic due to their exaggerated colors or lack of photographic noise.

Limitations

Despite the overall success, the project has several limitations:

- **Dataset Size and Diversity:** While the dataset was balanced, its overall size was moderate. Including more diverse AI-generated and real-world images would improve generalization.
- **Overfitting Risk:** With a relatively small training set and a powerful model like ResNet50, there's a risk of overfitting despite using validation splitting.
- **Domain Specificity:** The model might underperform on datasets with significantly different image distributions (e.g., medical images, satellite photos).
- **Binary Classification Limitation:** The model was trained for a binary classification task (AI vs. Human), and doesn't differentiate between different AI models (e.g., DALL·E vs. Midjourney).

Challenges Encountered

- **Library Compatibility:** Compatibility issues with packages (e.g., NumPy and scikit-learn) caused installation problems and debugging delays.
- **MPS Backend Stability:** While the MPS (Metal Performance Shaders) backend allowed GPU acceleration on macOS, certain PyTorch operations were not supported or had inconsistent performance.
- **Image Format Inconsistencies:** Some downloaded images were in unsupported or corrupted formats, requiring manual filtering and exception handling during preprocessing.
- **Visualization Tools:** Integrating training metrics and visual outputs (confusion matrix, loss curves) required additional effort to support LaTeX-compatible exports.

Summary

The project achieved its core objective of building a lightweight, real-time AI detection system. The model's ability to distinguish between synthetic and real images with high confidence suggests strong practical potential, particularly in content verification, moderation, and digital forensics. Future enhancements could include more diverse training data, multi-class classification (for specific AI models), and deployment on cloud or mobile platforms.

Chapter 6

Conclusion and Future Work

Conclusion

This project successfully demonstrated the feasibility of using deep learning for detecting AI-generated images. By fine-tuning a pre-trained ResNet50 model on a curated dataset of real and synthetic images, we developed an end-to-end pipeline capable of making accurate binary classifications with high performance.

The system achieved a validation accuracy of 93.80%, with strong precision (98.42%) and a balanced F1-score (93.48%). These results validate the effectiveness of deep convolutional networks in modeling subtle patterns and artifacts that distinguish AI-generated images from real photographs. Furthermore, the project extended beyond model training by integrating the solution into a user-friendly Flask-based web interface, allowing for real-time image uploads and predictions.

Contributions

The key contributions of this project include:

- A fully functional deep learning pipeline built using PyTorch, from data loading and preprocessing to model training and evaluation.
- A robust binary classifier using ResNet50 for distinguishing AI-generated and human-captured images.

- Real-time web deployment using Flask, with CPU/GPU (MPS) device selection and batch-free image inference.
- Evaluation with standard metrics including accuracy, precision, recall, F1-score, and confusion matrix.

Future Work

While the results are promising, several avenues remain open for future improvement:

- **Dataset Expansion:** Including a larger and more diverse set of real and AI-generated images, especially from different domains (e.g., faces, landscapes, medical).
- **Multiclass Detection:** Extending the binary classification system to a multi-class setting that identifies the specific source model (e.g., DALL·E, Midjourney, DeepMedia).
- **Model Optimization:** Exploring more efficient architectures like EfficientNet, MobileNetV3, or Vision Transformers (ViTs) to reduce inference time for deployment on mobile or embedded systems.
- **Explainability Integration:** Incorporating techniques such as Grad-CAM or SHAP to provide visual justifications for the model's predictions, enhancing transparency and user trust.
- **Cloud/Edge Deployment:** Hosting the solution on a cloud service (e.g., AWS, Azure) or converting it to ONNX/TensorFlow Lite for scalable or on-device deployment.

In summary, this project contributes a strong foundation toward the automated detection of synthetic media and offers a practical baseline for future research and real-world application in digital forensics, journalism, and media authentication.

Chapter 7

References

1. Y. Wang *et al.*, “Efficientnet-b7 for deepfake detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 12, pp. 1234–1245, 2020.

2. L. Zhang *et al.*, “Detecting gan-generated images using cnns,” *Computer Vision and Pattern Recognition*, 2019.

3. L. Verdoliva, “Media forensics and deepfakes: an overview,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, no. 5, pp. 910–932, 2020.

4. B. Bayar and M. C. Stamm, “A deep learning approach to universal image manipulation detection using a new convolutional layer,” in *Proceedings of the 4th ACM Workshop on Information Hiding and Multimedia Security*, 2016, pp. 5–10.

5. N. Rahmouni, V. Nozick, A. Hadid, and M. Chen, “Distinguishing computer graphics from natural images using convolution neural networks,” in *2017 IEEE Workshop on Information Forensics and Security (WIFS)*. IEEE, 2017, pp. 1–6.

6. J. Frank, T. Eisenhofer, A. Fischer, G. J. Brostow, and S. Knopp, “Leveraging frequency analysis for deep fake image recognition,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 3247–3258.

7. F. Marra *et al.*, “Detection of gan-generated fake images using co-occurrence matrices,” in *IEEE WIFS*, 2018.

8. R. Durall *et al.*, “Watch your up-convolution: CNN based generative deep neural networks are failing to reproduce spectral distributions,” *CVPR Workshops*, 2020.

Chapter 8

Appendix

A. Model Training Code Snippet

Below is a simplified version of the ensemble model training pipeline using MLP, XGBoost, and Random Forest with a Logistic Regression meta-learner:

```
# MLP definition
mlp = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.2),
    Dense(2, activation='softmax')
])
mlp.compile(optimizer=Adam(learning_rate=0.0001),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
mlp.fit(X_train, y_train, epochs=30, batch_size=32,
        validation_data=(X_val, y_val), verbose=0)

# XGBoost and Random Forest
xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
```

```
xgb.fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)

# Meta-learner
meta_input = np.vstack([mlp_pred, xgb_pred, rf_pred]).T
meta_clf = LogisticRegression()
meta_clf.fit(meta_input, y_val)
```

B. Face Extraction from Video

```
def extract_faces_from_video(video_path, max_faces=60):
    detector = MTCNN()
    cap = cv2.VideoCapture(video_path)
    faces = []
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        detections = detector.detect_faces(rgb)
        for face in detections:
            x, y, w, h = face['box']
            face_crop = rgb[y:y+h, x:x+w]
            face_crop = cv2.resize(face_crop, (224, 224))
            faces.append(face_crop)
        if len(faces) >= max_faces: break
    return faces
```

C. Feature Extraction

```
def extract_features(faces):
    x_feats, e_feats = [], []
```

```
    for img in faces:
        x_input = xception_preprocess(np.expand_dims(img, axis=0))
        e_input = efficientnet_preprocess(np.expand_dims(img, axis=0))
        x_feat = xception_model.predict(x_input)[0]
        e_feat = efficientnet_model.predict(e_input)[0]
        fused = np.concatenate([x_feat, e_feat])
        x_feats.append(fused)
    return np.array(x_feats)
```

D. End-to-End Video Prediction Pipeline

```
def predict_video_fake_or_real(video_path):
    faces = extract_faces_from_video(video_path)
    features = extract_features(faces)
    mlp_preds = mlp.predict(features)
    mlp_classes = np.argmax(mlp_preds, axis=1)
    xgb_preds = xgb.predict(features)
    rf_preds = rf.predict(features)
    stacked_preds = np.vstack([mlp_classes, xgb_preds, rf_preds]).T
    final_preds = meta_clf.predict(stacked_preds)
    decision = Counter(final_preds).most_common(1)[0][0]
    return "Fake" if decision == 1 else "Real"
```

E. Additional Visualizations

- Confusion Matrix – saved as `confusion_matrix.png`
- Accuracy Comparison – saved as `accuracy_comparison.png`
- Precision-Recall Curve – saved as `precision_recall_curve.png`
- ROC Curve – saved as `roc_curve.png`